

1 2 3 4 5 6 7 8 9 10 11 12 13 14	<pre>--- title: "HW2" output: pdf_document date: "2025-03-18" --- ```{r setup, include=FALSE} knitr::opts_chunk\$set(echo = TRUE) ``` Given elastic-net optimization problem,</pre> $\min_{\beta} \ y - X\beta\ ^2 + \lambda (\alpha \ \beta\ _2^2 + (1 - \alpha) \ \beta\ _1).$	
15 16 17 18 19 20 21 22 23 24 25 26 27	<pre>where: - \$ y - X\beta ^2\$ is least squares loss function - \$\lambda\$ is a regularization parameter - \$\alpha \ \beta\ _2^2\$ is the ridge (L2) regularization term - \$(1 - \alpha) \ \beta\ _1\$ is the lasso (L1) regularization term The ridge (L2) regularization term is by:</pre> $\alpha \ \beta\ _2^2 = \ \sqrt{\lambda \alpha} \beta\ ^2.$	
28 29 30 31 32 33 34	<pre>This term imposes squared penalty on regression coefficients. Let,</pre> $D = \sqrt{\lambda \alpha} I.$	
35 36 37 38 39 40	<pre>we define new feature matrix \$X'\$ and target \$y'\$ as</pre> $X' = \begin{bmatrix} X \\ \sqrt{\lambda \alpha} I \end{bmatrix}, \quad y' = \begin{bmatrix} y \\ 0 \end{bmatrix}.$	
41 42 43 44 45 46	<pre>the target vector remains the same. The additional term in new feature matrix, will include the ridge penalty term, this makes sure that when we calculate leas square it will included. the least squares loss term becomes:</pre> $\ y' - X'\beta\ ^2.$	
47 48 49 50 51 52	<pre>With \$X'\$ and \$y'\$, the elastic-net optimization problem becomes as the ridge penalty is included in least square part,</pre> $\min_{\beta} \ y' - X'\beta\ ^2 + (1 - \alpha) \lambda \ \beta\ _1.$	
53 54 55 56 57	<pre>taking, \$\lambda' = (1 - \alpha) \lambda\$, we will have Lasso problem.</pre> $\min_{\beta} \ y' - X'\beta\ ^2 + \lambda' \ \beta\ _1.$	

$$\min_{\beta} \|y'' - X''\beta\|^2 + \lambda' \|\beta\|_1.$$

Problem 2

Given linear regression,

$$y = X\beta + \epsilon$$

where the columns of X are orthonormal, meaning: $X^T X = I$

The least squares estimator for

β is :

$$\hat{\beta} = X^T y$$

1. Best Subset Selection (Size M)

Best subset selection tries to minimize the least square error while selecting exactly M predictors:

$$\min_{\beta} \|y - X\beta\|^2, \quad \text{subject to } |\beta|_0 \leq M.$$

$$\min_{\beta} \|y - X\beta\|^2, \quad \text{subject to } |\beta|_0 \leq M.$$

The best subset method selects the top M largest absolute values and sets the rest to zero, so we will end up with only M most significant predictors.

$$\text{Best Subset (Size M) estimator} = \hat{\beta}_{\text{best}} = \arg \min_{\beta} \|y - X\beta\|^2 \text{ subject to } |\beta|_0 \leq M.$$

$$\text{BestSubset(Size M) estimator} = \hat{\beta}_j \cdot I \left(|\hat{\beta}_j| \geq |\hat{\beta}^{(M)}| \right).$$

2. Ridge Regression

Ridge regression problem,

$$\min_{\beta} \|y - X\beta\|^2 + \lambda \|\beta\|_2^2.$$

$$\min_{\beta} \|y - X\beta\|^2 + \lambda \|\beta\|_2^2.$$

After expanding Ridge problem becomes,

$$\min_{\beta} \|y - X\beta\|^2 + \lambda \|\beta\|_2^2 = y^T y - 2y^T X\beta + \beta^T X^T X\beta + \lambda \beta^T \beta$$

$$\min_{\beta} (y^T y - 2y^T X\beta + \beta^T X^T X\beta + \lambda \beta^T \beta)$$

For estimator,

$$\nabla_{\beta} (-2y^T X\beta + \beta^T X^T X\beta + \lambda \beta^T \beta)$$

$$\nabla_{\beta} (-2y^T X\beta + \beta^T X^T X\beta + \lambda \beta^T \beta)$$

Since $X^T X = I$, the gradient with respect to β is:

$$-2X^T y + 2X^T X\beta + 2\lambda \beta = 0$$

$$-2X^T y + 2X^T X\beta + 2\lambda \beta = 0$$

Now, we solve for beta,

$$(X^T X + \lambda I) \beta = X^T y.$$

105	Now, we solve for beta,	
106		
107	$(X^T X + \lambda I) \hat{\beta} = X^T y.$	
108		
109		$(X^T X + \lambda I) \hat{\beta} = X^T y.$
110		
111	Since $X^T X = I$, we get,	
112		
113	$(I + \lambda I) \hat{\beta} = X^T y.$	
114		
115		$(I + \lambda I) \hat{\beta} = X^T y.$
116	The Ridge estimator is,	
117		
118	$\hat{\beta}_j^{\text{Ridge}} = \frac{\hat{\beta}_j}{1 + \lambda}.$	
119		
120		$\hat{\beta}_j^{\text{Ridge}} = \frac{\hat{\beta}_j}{1 + \lambda}.$
121		
122		
123	3. LASSO Regression	
124		
125	LASSO problem:	
126		
127	$\min_{\beta} y - X\beta ^2 + \lambda \beta _1.$	
128		
129		$\min_{\beta} y - X\beta ^2 + \lambda \beta _1.$
130		
131		
132	Just like in previous problem, expanding terms and taking derivative, we get	
133		
134	$-2X^T y + 2X^T X \hat{\beta} + \lambda \text{sign}(\hat{\beta}) = 0.$	
135		
136		$-2X^T y + 2X^T X \hat{\beta} + \lambda \text{sign}(\hat{\beta}) = 0.$
137		
138	Since $X^T X = I$, the beta from first term and 2 beta from second combine to form the estimator as lasso adds penatly term,	
139		
140	$\hat{\beta}_j - \lambda \text{sign}(\hat{\beta}_j) = 0.$	
141		
142		$\hat{\beta}_j - \lambda \text{sign}(\hat{\beta}_j) = 0.$
143		
144	As lasso takes the coefficients to zero, we can write it like this,	
145		
146	$\hat{\beta}_j = \text{sign}(\hat{\beta}_j) \max(0, \hat{\beta}_j - \lambda).$	
147		
148		$\hat{\beta}_j = \text{sign}(\hat{\beta}_j) \max(0, \hat{\beta}_j - \lambda).$
149		
150	Thus, LASSO estimator is,	
151		
152	$\hat{\beta}_j = \text{sign}(\hat{\beta}_j) (\hat{\beta}_j - \lambda)_+.$	
153		
154		$\hat{\beta}_j = \text{sign}(\hat{\beta}_j) (\hat{\beta}_j - \lambda)_+.$

```

155
156 Problem 3
157 ```{r}
158 library(glmnet)
159 library(boot)
160
161 set.seed(42)
162
163 num_rows <- 800
164 num_cols <- 200
165 num_bootstraps <- 1000
166
167 data_matrix <- matrix(rnorm(num_rows * num_cols), nrow = num_rows, ncol = num_cols)
168 coeffs <- 1 / (1:num_cols)
169 noise <- rnorm(num_rows)
170 target <- data_matrix %*% coeffs + noise
171
172 bootstrap_function <- function(dataset, sample_indices, method) {
173   sample_X <- dataset[sample_indices, -ncol(dataset)]
174   sample_y <- dataset[sample_indices, ncol(dataset)]
175
176   if (method == "OLS") {
177     simple_model <- lm(sample_y ~ sample_X - 1)
178     return(coef(simple_model)[1])
179   }
180
181   if (method == "ridge") {
182     ridge_model <- cv.glmnet(sample_X, sample_y, alpha = 0, lambda = 10^seq(-3, 3, length = 100))
183     best_ridge <- coef(ridge_model, s = "lambda.min")[-1]
184     return(best_ridge[1])
185   }
186
187   if (method == "lasso") {
188     lasso_model <- cv.glmnet(sample_X, sample_y, alpha = 1, lambda = 10^seq(-3, 3, length = 100))
189     best_lasso <- coef(lasso_model, s = "lambda.min")[-1]
190     return(best_lasso[1])
191   }
192 }
193
194 dataset <- cbind(data_matrix, target)
195
196 bootstrap_OLS <- boot(dataset, statistic = function(d, i) bootstrap_function(d, i, "OLS"), R = num_bootstraps)
197 CI_OLS <- boot.ci(bootstrap_OLS, type = "perc")$percent[4:5]

```

```

196 bootstrap_OLS <- boot(dataset, statistic = function(d, i) bootstrap_function(d, i, "OLS"), R = num_bootstraps)
197 CI_OLS <- boot.ci(bootstrap_OLS, type = "perc")$percent[4:5]
198
199 bootstrap_ridge <- boot(dataset, statistic = function(d, i) bootstrap_function(d, i, "ridge"), R = num_bootstraps)
200 CI_ridge <- boot.ci(bootstrap_ridge, type = "perc")$percent[4:5]
201
202 bootstrap_lasso <- boot(dataset, statistic = function(d, i) bootstrap_function(d, i, "lasso"), R = num_bootstraps)
203 CI_lasso <- boot.ci(bootstrap_lasso, type = "perc")$percent[4:5]
204
205 cat("Least Squares: [", CI_OLS[1], ", ", CI_OLS[2], "]\n")
206 cat("Ridge : [", CI_ridge[1], ", ", CI_ridge[2], "]\n")
207 cat("Lasso : [", CI_lasso[1], ", ", CI_lasso[2], "]\n")
208 ```

```

```

Least Squares: [ 0.8484409 , 1.044108 ]
Ridge : [ 0.6727431 , 0.8530534 ]
Lasso : [ 0.8226443 , 0.9821322 ]

```

209 Applying Least Squares to bootstrapped data gave a confidence interval of [0.8484, 1.0441]. It is wide, showing higher variance in estimates.
 210 Ridge Regression gave a narrower confidence interval of [0.6727, 0.8531]. As ridge applies L₂ penalty, it results in less variance, but introduces bias.
 211 Lasso Regression gave a confidence interval of [0.8226, 0.9821], which is narrower than Least Squares but wider than Ridge as expected as lasso forces some coefficients to be exactly zero.
 212 Ridge Regression gave the most stable estimates, useful we need low variance. Lasso is useful when feature selection is needed.

```

213
214 Problem 5
215 ```{r}
216 #part a
217 library(ISLR)
218 library(MASS)
219 library(class)
220 library(e1071)
221 library(ggplot2)
222
223 data(Auto)
224
225 Auto$mpg01 <- ifelse(Auto$mpg > median(Auto$mpg), 1, 0)
226
227 pairs(Auto[, c("mpg01", "displacement", "horsepower", "weight", "acceleration")],
228   main="Scatterplot Matrix of Selected Features")
229
230 #part b
231 boxplot(Auto$weight ~ Auto$mpg01,
232   main="Weight vs mpg01",
233   xlab="mpg01 (0 = Low, 1 = High)",
234   ylab="Weight")
235 #part c

```

```

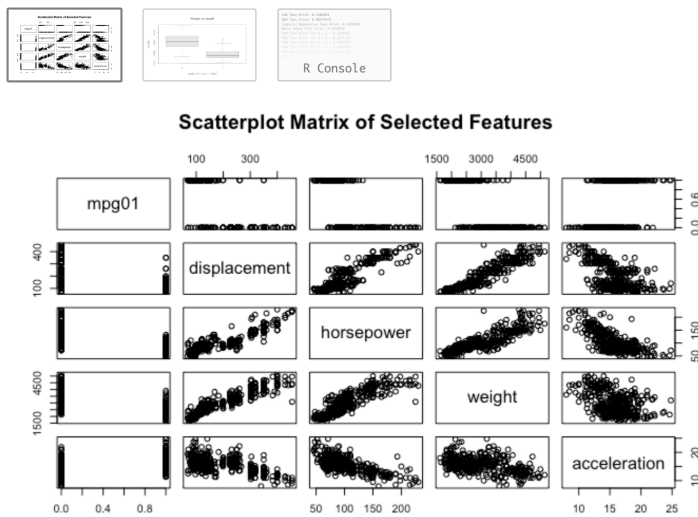
235 #part c
236 set.seed(123)
237 train_idx <- sample(1:nrow(Auto), 0.7 * nrow(Auto))
238 train_set <- Auto[train_idx, ]
239 test_set <- Auto[-train_idx, ]
240
241 features <- c("displacement", "horsepower", "weight", "acceleration")
242
243 #part d
244 lda_model <- lda(mpg01 ~ displacement + horsepower + weight + acceleration, data=train_set)
245 lda_pred <- predict(lda_model, test_set)
246 lda_err <- mean(lda_pred$class != test_set$mpg01)
247 cat("LDA Test Error:", lda_err, "\n")
248
249 #part e
250 qda_model <- qda(mpg01 ~ displacement + horsepower + weight + acceleration, data=train_set)
251 qda_pred <- predict(qda_model, test_set)
252 qda_err <- mean(qda_pred$class != test_set$mpg01)
253 cat("QDA Test Error:", qda_err, "\n")
254
255 #part f
256 logistic_model <- glm(mpg01 ~ displacement + horsepower + weight + acceleration,
257                       data=train_set, family=binomial)
258 logistic_probs <- predict(logistic_model, test_set, type="response")
259 logistic_pred <- ifelse(logistic_probs > 0.5, 1, 0)
260 logistic_err <- mean(logistic_pred != test_set$mpg01)
261 cat("Logistic Regression Test Error:", logistic_err, "\n")
262
263 #part g
264 nb_model <- naiveBayes(mpg01 ~ displacement + horsepower + weight + acceleration, data=train_set)
265 nb_pred <- predict(nb_model, test_set)
266 nb_err <- mean(nb_pred != test_set$mpg01)
267 cat("Naive Bayes Test Error:", nb_err, "\n")
268
269 train_x <- train_set[, features]
270 test_x <- test_set[, features]
271 train_y <- train_set$mpg01
272 test_y <- test_set$mpg01
273
274 #part h
275 knn_errs <- c()
276 for (k in c(1, 3, 5, 7, 9)) {
277   knn_pred <- knn(train_x, test_x, train_y, k=k)
278   err <- mean(knn_pred != test_y)
279   knn_errs <- c(knn_errs, err)
280 }

```

```

280 cat("KNN Test Error for k =", k, ":", err, "\n")
281 ~ }
282
283 best_k <- c(1, 3, 5, 7, 9)[which.min(knn_errs)]
284 cat("Best K:", best_k, "\n")
285 ~`

```



286 The boxplot of Weight vs. mpg01 clearly indicates that lighter cars tend to have higher mileage. The scatterplot shows strong correlation between mpg01 and variables such as displacement, horsepower, and weight, these features are most likely to be useful in predicting mpg01.

287 The results indicate that QDA performed the best, with the lowest test error of 0.0847, suggesting that a nonlinear boundary captures relationship between the predictors and mpg01 the best. LDA has a slightly higher test error 0.1186, which implies that a linear boundary is somewhat effective, but it does not fully capture the relationship. Logistic Regression and Naive Bayes have similar results with test errors of 0.1017, this indicates that both models distinguish between high and low mileage cars well.

288 Their comparable performance suggests that the assumption of conditional independence in Naive Bayes does not significantly impact accuracy for this dataset.

289 KNN performance varies with the choice of K, K=7 gave the lowest test error (0.1017), which is similar to Logistic Regression and Naive Bayes.

290 However, lower values of K, such as K=1 (0.1695 test error), resulted in overfitting, while larger K values caused a slight increase in test error.

